GENERATIVE DEEP LEARNING | CHAPTER 2

Deep Learning

Build a working supervised neural network before using neural networks as generators. Understand data shapes, nonlinear layers, objectives, optimization, regularization, and evaluation.

Original study notes based on David Foster, Second edition, 2023. Book pp. 23–58. Source APIs, services, experiments, and model comparisons reflect the book's publication period. These are explanatory summaries, not verbatim reproductions or a claim that every source detail is retained.

Prepare data → Build layers → Choose loss and optimizer → Train → Evaluate held-out examples

Detailed Notes

Data and high-level feature learning

Source: book pp. 23–27 / PDF p. 51

Images, text, and audio have structure that raw scalar values do not express directly. Deep networks learn successively useful representations through stacked transformations. Depth without nonlinear activation would collapse into a single linear transformation.

Separate features from labels and training from evaluation data. Inspect shapes, ranges, class frequencies, and sample examples before fitting. TensorFlow supplies tensor computation and differentiation; Keras supplies model/layer abstractions. Sequential models express simple stacks, while the functional API handles shared layers, multiple inputs/outputs, and nontrivial graphs.

MLP project: preparing CIFAR-style image data

Source: book pp. 28–29 / PDF p. 56

The chapter's image-classification project starts by normalizing pixel intensities and encoding the class labels. Preserve the correspondence among image channels, image dimensions, and label indices. An MLP flattens the image, losing explicit spatial locality but providing a straightforward baseline.

Fit any learned preprocessing on training data only. Convert labels into the form expected by the objective: integer labels for sparse categorical loss or one-hot vectors for categorical loss. Mixing these conventions can produce shape errors or incorrect training.

Building, compiling, and training an MLP

Source: book pp. 30–37 / PDF p. 58

A dense layer computes a learned affine transform followed by an activation. ReLU-like activations provide nonlinearity; the final softmax turns class logits into probabilities summing to one. The number of output units matches the class count.

> $h = \text{activation}(Wx+b)$
> $\text{Cross-entropy} = -\Sigma_{\text{classes}}\, y_c \log p_c$

Compilation configures loss, optimizer, and metrics. Backpropagation computes gradients; an optimizer such as Adam updates weights. Batch size controls examples per update, epochs count passes, and learning rate governs update scale. Monitor validation behavior rather than choosing an epoch count solely from decreasing training loss.

Evaluating the model

Source: book pp. 38–40 / PDF p. 66

Evaluate on data not used to optimize weights or choose hyperparameters. Compare predicted classes and actual labels, inspect errors, and distinguish confidence from correctness. Accuracy summarizes classification success but can hide class-specific failures.

Training accuracy substantially above validation accuracy suggests overfitting, though label noise and distribution mismatch can also matter. Underfitting can arise from inadequate capacity, optimization, or representation. Keep the final test set untouched while experimenting, and use a simple baseline to establish whether added complexity helps.

Convolutional layers and tensor geometry

Source: book pp. 41–45 / PDF p. 69

Convolutions share small kernels across spatial locations. A kernel sees local patches; multiple filters learn different patterns; stacked layers enlarge the effective receptive field. Stride and padding determine output spatial size, while filters determine output channels.

> $\text{Output size} = \text{floor}((\text{input} + 2\times\text{padding} - \text{kernel})/\text{stride}) + 1$
> $\text{Conv parameters} = (\text{kernel height} \times \text{kernel width} \times \text{input channels} + \text{bias})\times \text{filters}$

Track shapes after each layer instead of guessing. Weight sharing typically needs far fewer parameters than a dense layer over all pixels. Downsampling reduces spatial size but can discard fine details.

Batch normalization and dropout

Source: book pp. 46–50 / PDF p. 74

Batch normalization normalizes activations using batch statistics during training, then learned scale/shift parameters. Inference usually uses moving statistics. Training and inference modes therefore behave differently, especially with very small or unrepresentative batches.

Dropout randomly disables units during training to reduce co-adaptation; it is normally disabled for inference, with the appropriate scaling convention. Neither technique fixes bad labels or leakage. Placement, rates, batch size, and model family matter; later adversarial models require particular care with normalization and training-mode switches.

Building, training, and evaluating the CNN

Source: book pp. 51–58 / PDF p. 79

Combine convolutional feature extraction with activations, normalization, regularization, and a final classification head. Compare its held-out performance with the MLP under a consistent split and measurement protocol.

Inspect both parameter counts and runtime. A CNN's spatial inductive bias can help images, but architectural sophistication is not evidence of improvement by itself. Retain checkpoints and preprocessing details so sampling or classification after reloading behaves the same. These building blocks recur in the encoder, decoder, generator, critic, and denoiser architectures that follow.

Revision Check

1. Why are nonlinear activations essential?
2. How do stride and padding change convolution output size?
3. What changes between training and inference in batch normalization?
4. Which observations distinguish overfitting from underfitting?

Know each tensor's meaning and shape, and each objective's target. These fundamentals make generative architectures debuggable.

Source links refer to the supplied PDF. Every contents-page topic is mapped in coverage.md. Consult the book for complete implementations, figures, exercises, citations, and experiment evidence.